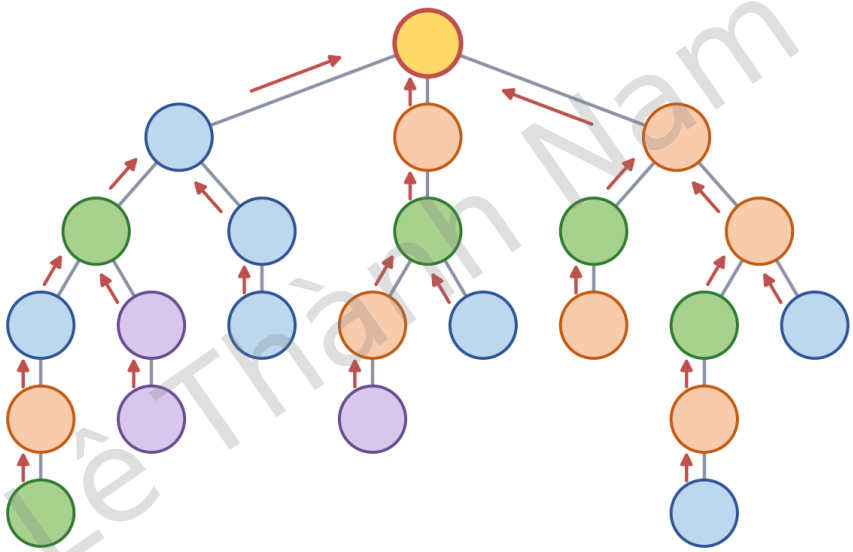
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

THUẬT TOÁN TRÊN CÂY

Dễ hiểu cho người bắt đầu từ con số 0

TẬP 4 · KỸ THUẬT GỘP



Lê Thành Nam

Gộp nhỏ vào lớn · DSU on tree · Gộp cây phân đoạn · Heap trái · Heap lệch · Heap ghép cặp

Chương 1. GỘP NHỎ VÀO LỚN
Chương 2. DSU ON TREE (SACK)
Chương 3. GỘP CÂY PHÂN ĐOẠN
Chương 4. HEAP GỘP ĐƯỢC

LỜI NÓI ĐẦU

Rất nhiều bài toán trên cây có chung một hình dạng: thông tin của một cây con là **một tập hợp** (tập màu, tập giá trị, bảng đếm, hàng đợi ưu tiên), và thông tin của cha là hợp thông tin của các con. Làm thẳng tay thì mỗi phần tử bị chép lên mọi tổ tiên: $O(n^2)$. Cuốn sách này dạy các **kỹ thuật gộp**: những cách gộp thông tin của các cây con sao cho tổng chi phí chỉ còn $O(n \log n)$, cùng một lập luận chung để chứng minh điều đó: lập luận **nhân đôi**.

Đây là **Tập 4: Kỹ thuật gộp** của bộ sách về thuật toán trên cây, tiếp nối Tập 1 (Nền tảng), Tập 2 (Tổ tiên chung gần nhất) và Tập 3 (Quy hoạch động trên cây). Em cần đã quen với: thứ tự BFS và Euler tour, kích thước cây con, con nặng và cạnh nhẹ, cây phân đoạn, DSU. Những kiến thức ấy được nhắc lại ở Phần mở đầu.

Giống các tập trước, mỗi bài gồm hai phần:

Phần A – Lí thuyết: định nghĩa, định lí kèm chứng minh từng ý, thuật toán, chạy tay từng bước trên hình (phần lớn trên cây mẫu T14), code mẫu, độ phức tạp, bẫy thường gặp.

Phần B – Bài vận dụng: một bài toán theo khuôn đề thi, giải đầy đủ: ý tưởng ngây thơ với sơ đồ khối và chương trình vét cạn, chỉ ra bế tắc, áp dụng lí thuyết để phá bế tắc, chạy tay, chương trình hoàn chỉnh, mở rộng và luyện tập.

Sách gồm **Phần mở đầu** và **10 bài học chia thành 4 chương**:

Chương 1. Gộp nhỏ vào lớn: set, tập có thứ hạng (pb_ds), map; chứng minh $O(n \log n)$.

Chương 2. DSU on tree (Sack): một mảng đếm toàn cục cho mọi cây con; theo màu và theo độ sâu.

Chương 3. Gộp cây phân đoạn: cây phân đoạn động, phép gộp $O(n \log m)$, số nhỏ thứ k, đếm nghịch thế.

Chương 4. Heap gộp được: heap trái, heap lệch, heap ghép cặp; kết hợp DSU; Dijkstra có giảm khóa.

Các bài vận dụng gồm những bài kinh điển: CSES Distinct Colors (Bài 1), USACO Promotion Counting (Bài 2), Codeforces 600E Lomsat gelral (Bài 3), Codeforces 375D Tree and Queries (Bài 4), Codeforces 570D Tree Requests (Bài 5), POI 2011 Tree Rotations (Bài 7), Monkey King (Bài 9). Mọi chương trình đều đã được đối chiếu với chương trình vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên và chạy thử với dữ liệu lớn nhất; mọi ví dụ trong sách đều được chạy lại bằng chính các chương trình đó.

Chương trình viết bằng C++17, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp", dùng được với Themis lẫn các trang chấm trực tuyến. Mọi chương trình đều không có chú thích bên trong; các dòng quan trọng được giải thích ngay bên dưới.

Chúc các em và các thầy cô học tập say mê, dạy tốt và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học có cùng một cấu trúc:

Phần	Nội dung
Tóm tắt bài học	Khung tóm tắt ở đầu bài: kiến thức chính, khuôn cài đặt, độ phức tạp, khi nào dùng, bài vận dụng.
Phần A – Lí thuyết	Định nghĩa, tính chất, định lí kèm chứng minh từng ý; thuật toán; chạy tay từng bước nhỏ trên hình; code mẫu; độ phức tạp; bẫy; ghi nhớ.
Phần B – Bài vận dụng	Đề bài như đề thi thật: dữ liệu vào, kết quả, giới hạn, ví dụ.
B1. Ý tưởng ngây thơ	Cách làm ai cũng nghĩ ra: sơ đồ khối, chạy tay, chương trình vét cạn, đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ quá chậm hoặc quá tốn bộ nhớ.
B2. Phá bế tắc	Áp dụng lí thuyết Phần A, từng bước, chạy tay trên hình.
B3. Chương trình hoàn chỉnh	Chương trình C++ đã kiểm chứng, giải thích các dòng quan trọng.
B4. Mở rộng, Luyện tập	Các biến thể thường gặp; bài luyện thêm và câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
TÓM TẮT BÀI HỌC	Tóm tắt toàn bộ bài ở ngay đầu bài.
ĐỊNH NGHĨA	Định nghĩa chính xác một khái niệm.
TÍNH CHẤT / ĐỊNH LÍ	Một khẳng định quan trọng, luôn có chứng minh đi kèm.
THUẬT TOÁN	Các bước của một thuật toán.
HỘP CÔNG CỤ C++	Giới thiệu một công cụ C++ khi nó được dùng lần đầu.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp: vì sao chậm, vì sao nhanh.
BẾ TẮC	Chỗ cách làm ngây thơ không vượt qua được.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối phần lí thuyết.

Quy ước màu trong các hình được giữ nguyên trong cả cuốn sách: **đỉnh vàng viền đỏ** là đỉnh đang xét; **đỉnh xanh lá** là đỉnh đã xử lí xong, hoặc đỉnh được chọn trong lời giải; **đỉnh xanh dương nhạt** là đỉnh bình thường; **đỉnh đỏ nhạt** là đỉnh bị loại hoặc vi phạm điều kiện; **gốc** tô xanh đậm. Ở Chương 1, 2, màu nền của đỉnh là màu c[v] của đỉnh đó. Chữ nhỏ cạnh mỗi đỉnh là dữ liệu đang tính (tập, bảng đếm, giá trị); cạnh đỏ đậm nối với con nặng hoặc là nhánh phải của heap; ô hình chữ nhật là nút của cây phân đoạn.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Gặp một hình chạy tay, hãy che phần lời giải lại, tự làm bước đó rồi mới đọc. Gặp một định lí, hãy tự vẽ một cái cây khác và kiểm

Lê Thành Nam

tra định lí trên cây của mình. Chỉ khi tự viết lại được chương trình và tự giải thích được vì sao nó đúng mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**
CÁCH HỌC VỚI TÀI LIỆU NÀY **3**
PHẦN MỞ ĐẦU. KỸ THUẬT GỘP VÀ BẢN ĐỒ TẬP 4 ..... **6**
CHƯƠNG 1. GỘP NHỎ VÀO LỚN **9**
Bài 1. Gộp nhỏ vào lớn với set 10
Bài 2. Gộp tập có thứ hạng 26
Bài 3. Gộp map và đại lượng đi kèm 39
CHƯƠNG 2. DSU ON TREE (SACK) **50**
Bài 4. DSU on tree (Sack) 51
Bài 5. DSU on tree theo độ sâu 66
CHƯƠNG 3. GỘP CÂY PHÂN ĐOẠN **77**
Bài 6. Gộp cây phân đoạn 78
Bài 7. Gộp cây phân đoạn có đếm 93
CHƯƠNG 4. HEAP GỘP ĐƯỢC **104**
Bài 8. Heap trái 105
Bài 9. Heap lệch 117
Bài 10. Heap ghép cặp 128
TỔNG KẾT: CHỌN KỸ THUẬT GỘP **141**

Lê Thành Nam

PHẦN MỞ ĐẦU

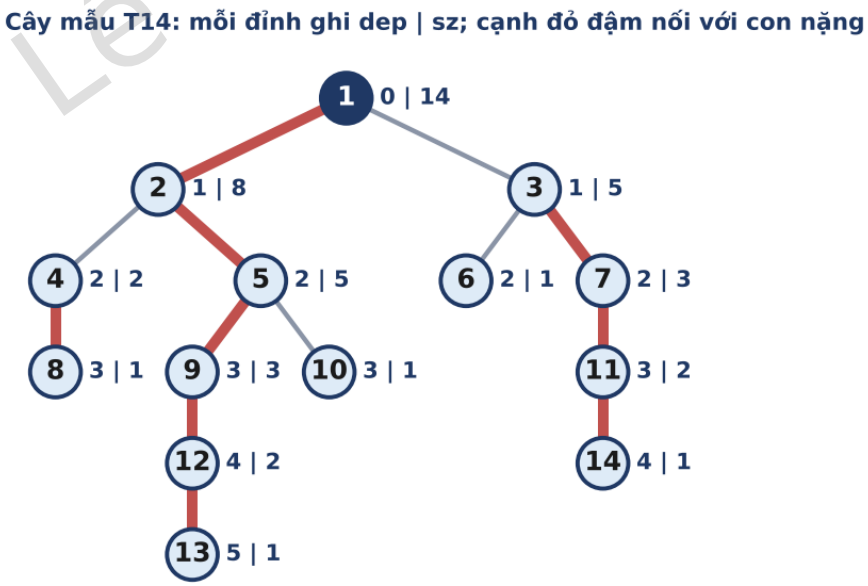
KỸ THUẬT GỘP VÀ BẢN ĐỒ TẬP 4

Phần này nhắc lại các kí hiệu từ ba tập trước, giải thích vấn đề chung mà mọi kỹ thuật trong cuốn sách cùng giải quyết (gộp thông tin của các cây con lên cha), trình bày lập luận "nhân đôi" xuyên suốt, và vẽ bản đồ bốn chương.

1. Nhắc lại

Cây được treo ở **gốc 1**. Các kí hiệu dùng xuyên suốt:

Kí hiệu	Ý nghĩa	Ở đâu
par[v], ord	cha; thứ tự BFS (cha trước con)	Tập 1
T(v), sz[v]	cây con gốc v; số đỉnh của nó	Tập 1
tin[v], ver[]	thứ tự trước; T(v) = ver[tin[v] .. tin[v] + sz[v] − 1]	Tập 1
big[v]	con nặng: con có sz lớn nhất	Tập 2
cạnh nhẹ	cạnh (v, u) với u ≠ big[v]; sz[u] ≤ sz[v]/2	Tập 2
"đẩy lên cha"	duyệt ngược BFS, xử lí xong u thì gộp u vào cha	Tập 3



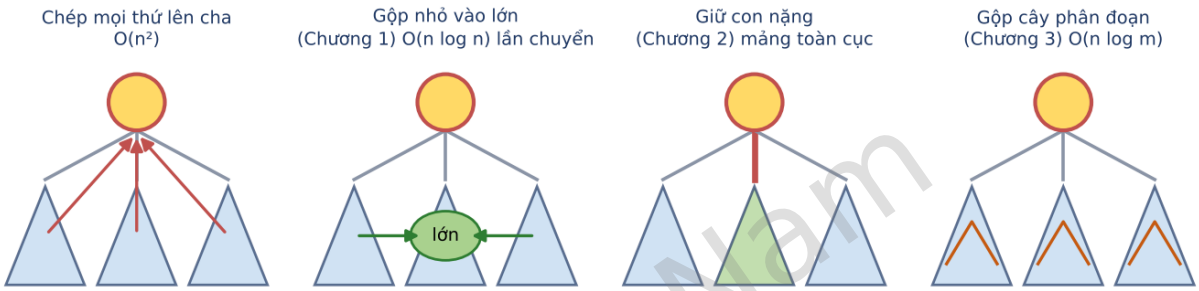
Hình 0.1. *Cây mẫu T14*

Lê Thanh Nam

Các ví dụ chạy tay dùng lại cây T14 (danh sách cạnh: 1 2, 1 3, 2 4, 2 5, 3 6, 3 7, 4 8, 5 9, 5 10, 7 11, 9 12, 12 13, 11 14). Ở Chương 1 và 2, mỗi đỉnh có thêm một màu (1 xanh dương, 2 xanh lá, 3 cam, 4 tím).

2. Bài toán chung: gộp thông tin của các cây con

Ở Tập 3, thông tin của T(v) gói trong vài số, hay một mảng nhỏ. Tập này xét các bài mà thông tin của T(v) là **cả một tập hợp**: tập các màu, tập các giá trị có thứ tự, bảng đếm số lần, một hàng đợi ưu tiên, ... Thông tin của cha là **hợp** thông tin của các con (cộng thêm chính nó). Câu hỏi: làm sao tính hợp ấy cho **mọi** đỉnh mà không tốn $O(n^2)$?



Hình 0.2. *Bốn cách gộp thông tin của các con lên cha*

- **Chép mọi thứ lên cha:** mỗi phần tử bị chép một lần ở mỗi tổ tiên: $O(n^2)$ với cây đường thẳng.
- **Gộp nhỏ vào lớn (Chương 1):** giữ lại cấu trúc lớn nhất, chèn các cấu trúc nhỏ vào.
- **Giữ con nặng (Chương 2):** chỉ có một mảng toàn cục; giữ dữ liệu của con nặng, làm lại các con nhẹ.
- **Gộp cấu trúc có hình dạng phù hợp (Chương 3, 4):** cây phân đoạn động, các heap gộp được: hai cấu trúc ghép với nhau nhanh hơn chèn từng phần tử.

3. Lập luận nhân đôi

> **BỔ ĐỀ NHÂN ĐÔI**
>
> Nếu mỗi lần một phần tử x bị "trả phí" thì kích thước của nhóm chứa x **ít nhất gấp đôi**, thì x bị trả phí **không quá $\log_2 n$ lần**. Tổng chi phí của n phần tử là **$O(n \log n)$**.

Chứng minh: nhóm chứa x bắt đầu từ kích thước ≥ 1 và không vượt quá n; sau k lần gấp đôi kích thước $\geq 2^k$, nên $2^k \leq n$, $k \leq \log_2 n$.

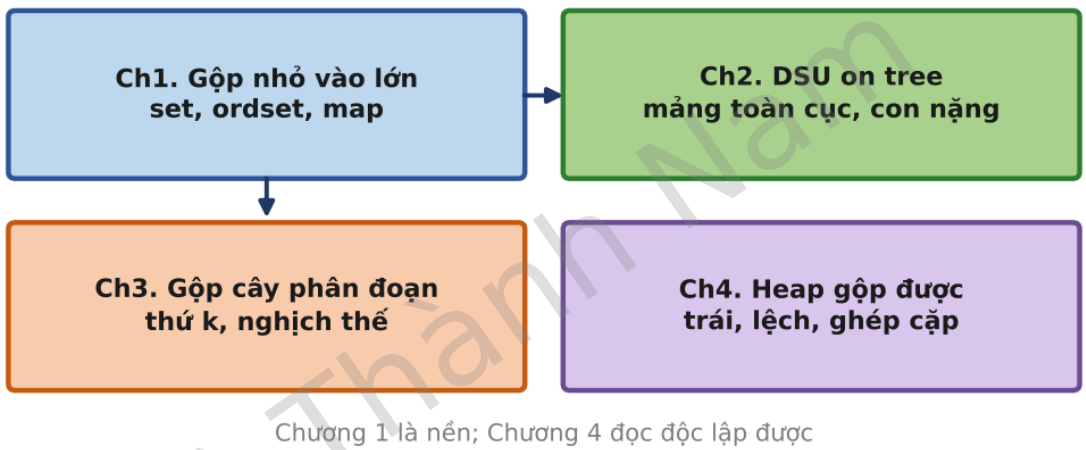
Bổ đề này xuất hiện dưới nhiều dạng: gộp nhỏ vào lớn (phần tử bị chuyển khi nằm ở tập nhỏ hơn), cạnh nhẹ (đi qua cạnh nhẹ thì sz ít nhất gấp đôi), heap trái (nhánh phải có d + 1 nút thì cây có $\geq 2^{d+1} - 1$ nút), nút nhẹ trong heap lệch (đi sang phải từ nút nhẹ, kích thước giảm một nửa). Khi gặp một kỹ thuật gộp mới, câu hỏi đầu tiên nên là: **"cái gì gấp đôi?"**.

Lê Thành Nam

4. Chọn kỹ thuật

Đề hỏi	Kỹ thuật	Bài
tập/đếm trên cây con, cấu trúc phức tạp (set, map, ordset)	gộp nhỏ vào lớn	1, 2, 3
câu hỏi ngoại tuyến trên cây con, đếm bằng mảng	DSU on tree	4, 5
thống kê theo giá trị (thứ k, đếm trong khoảng), đếm cặp chéo	gộp cây phân đoạn	6, 7
gộp nhóm và lấy nhỏ nhất/lớn nhất	heap trái, heap lệch	8, 9
hàng đợi ưu tiên cần giảm khóa	heap ghép cặp	10

5. Bản đồ Tập 4



Hình 0.3. *Bốn chương*

- **Chương 1 (Bài 1–3):** gộp nhỏ vào lớn với set, tập có thứ hạng, map; chứng minh O(n log n) kể cả khi có phần tử trùng.
- **Chương 2 (Bài 4–5):** DSU on tree; mảng theo màu, mảng theo độ sâu; cài không đệ quy.
- **Chương 3 (Bài 6–7):** cây phân đoạn động, phép gộp O(n log m), số nhỏ thứ k, đếm nghịch thế chéo.
- **Chương 4 (Bài 8–10):** heap trái, heap lệch (khấu hao), heap ghép cặp (giảm khóa), áp dụng cho DSU và Dijkstra.

> **KHÔNG ĐỆ QUY**
> Như các tập trước, mọi duyệt cây đều không đệ quy. Có hai chỗ dùng đệ quy có chủ ý, vì độ sâu chỉ là O(log): phép gộp cây phân đoạn (độ sâu log m), phép gộp heap trái (độ sâu ≤ 2 log n). Heap lệch thì **bắt buộc** cài không đệ quy (Bài 9).

Lê Thành Nam

6. Đối chiếu chương trình

Mọi chương trình chính đã được đối chiếu với chương trình trâu trên 500 bộ dữ liệu nhỏ ngẫu nhiên (cây có năm dạng: ngẫu nhiên, đường thẳng, hình sao, "con sâu", cây sâu), và chạy thử với dữ liệu lớn nhất; thời gian đo được ghi trong từng bài. Mọi ví dụ trong sách đều được chạy lại bằng chính các chương trình đó.

Lê Thành Nam

CHƯƠNG 1. GỘP NHỎ VÀO LỚN

Tập của cha là hợp các tập của con. Đừng chép tất cả lên cha: hãy giữ lại tập lớn nhất và chỉ chèn các tập nhỏ vào nó. Mỗi phần tử bị chuyển thì tập chứa nó ít nhất gấp đôi, nên không phần tử nào bị chuyển quá log n lần.

TRONG CHƯƠNG NÀY

Bài 1. Gộp nhỏ vào lớn với set *(vận dụng: số màu trong cây con (CSES Distinct Colors))*

Bài 2. Gộp tập có thứ hạng *(vận dụng: đếm cấp dưới giỏi hơn (USACO Promotion Counting))*

Bài 3. Gộp map và đại lượng đi kèm *(vận dụng: màu trội (CF 600E Lomsat gelral))*

Lê Thành Nam

BÀI 1 Gộp nhỏ vào lớn với set

Bài vận dụng: Số màu trong cây con (CSES Distinct Colors)

TÓM TẮT BÀI 1. GỘP NHỎ VÀO LỚN VỚI SET

Chủ đề: Gộp nhỏ vào lớn

Kiến thức chính:

- S(v) = {c[v]} ∪ hợp các S(con); chép tập con lên cha tốn $O(n^2)$.
- **Giữ tập lớn, chèn tập nhỏ**: mỗi phần tử bị chuyển thì tập chứa nó ít nhất gấp đôi ⇒ $\leq \log_2 n$ lần.
- Có phần tử trùng vẫn O(n log n) (đếm theo số đỉnh đại diện).
- Cài bằng id[v] + đổi chỗ O(1); duyệt ngược BFS; giải phóng tập đã dùng.

Khuôn cài đặt:

```
ans[u] = s[id[u]].size();
int a = id[p], b = id[u];
if (s[a].size() < s[b].size()) swap(a, b);
for (int x : s[b]) s[a].insert(x);
set<int>().swap(s[b]); id[p] = a;
```

Độ phức tạp: $O(n \log^2 n)$ với set

Dùng khi: Thông tin cây con là một tập, gộp bằng hợp

Bài vận dụng: Số màu trong cây con (CSES Distinct Colors) [SOMAU]

Lê Thanh Nam

Học xong bài này, em sẽ:

- ✓ Nhận ra dạng bài **"tập các giá trị trong cây con"**: thông tin của T(v) là hợp các tập của các cây con.
- ✓ Nắm kĩ thuật **gộp nhỏ vào lớn** (small-to-large): luôn chèn tập nhỏ vào tập lớn, giữ lại tập lớn.
- ✓ Chứng minh mỗi phần tử bị chuyển **không quá $\log_2 n$ lần**, tổng O(n log n) lần chèn, kể cả khi các tập có phần tử trùng nhau.
- ✓ Cài đặt bằng **mảng chỉ số id** và phép đổi chỗ O(1), duyệt ngược BFS, không đệ quy.

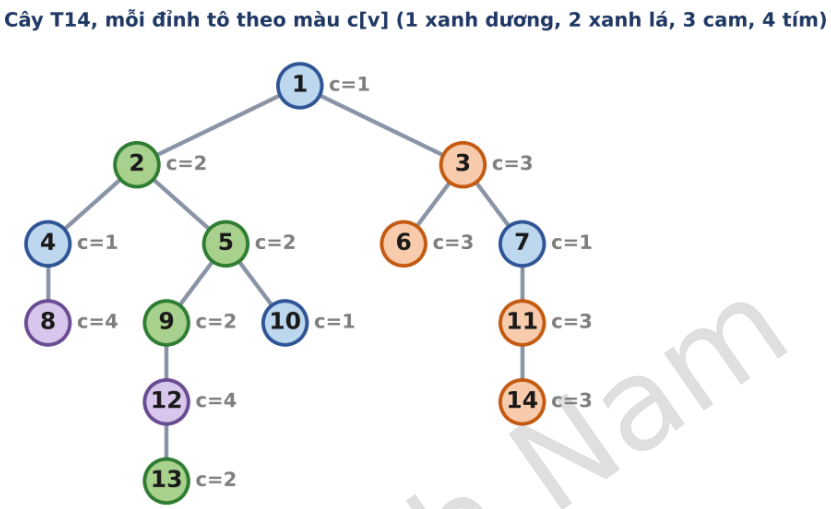
PHẦN A LÍ THUYẾT

A1. Tập giá trị của một cây con

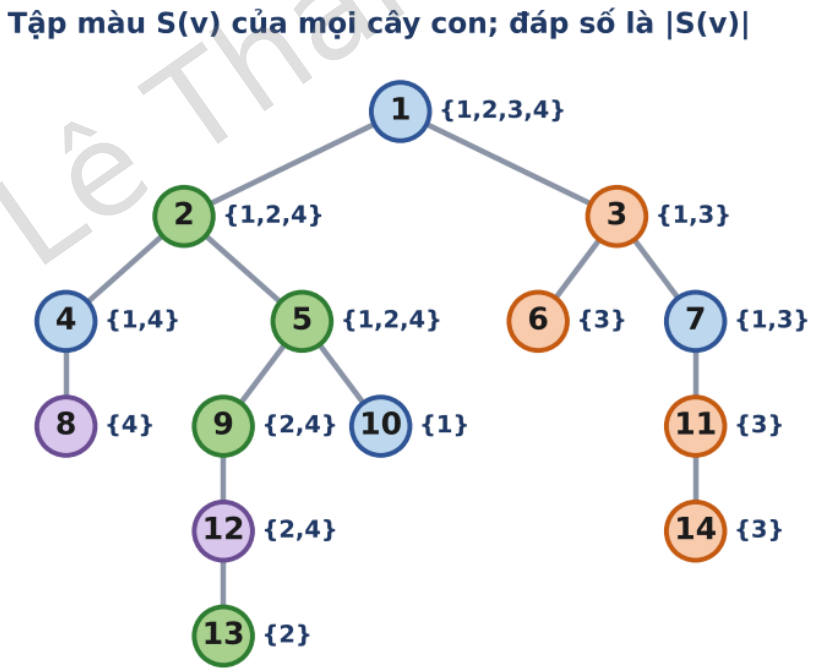
Trong cả tập sách, cây được treo ở **gốc 1**; T(v) là cây con gốc v. Mỗi đỉnh v có một **màu** c[v] (một số nguyên bất kì).

ĐỊNH NGHĨA

- **S(v) = { c[x] : x ∈ T(v) }**: tập các màu xuất hiện trong cây con gốc v (mỗi màu chỉ tính một lần).
- Bài toán: tính **|S(v)|** (số màu khác nhau trong T(v)) cho **mọi** đỉnh v.



Hình 1.1. *Cây T14 với màu của các đỉnh*



Hình 1.2. *Tập màu của mọi cây con*

Lê Thành Nam

CÔNG THỨC HỢP

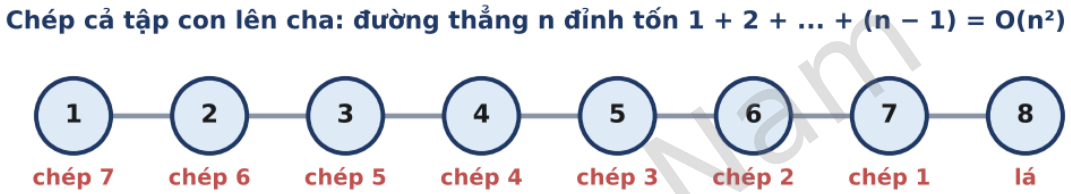
$S(v) = \{c[v]\} \cup S(c_1) \cup S(c_2) \cup \ldots \cup S(c_k)$, với $c_1, \ldots, c_k$ là các con của v.

Lí do: $T(v)$ gồm v và các cây con $T(c_i)$ rời nhau, nên tập màu của $T(v)$ là hợp các tập màu của từng phần.

Trên Hình 1.2: $S(5) = \{2\} \cup S(9) \cup S(10) = \{2\} \cup \{2, 4\} \cup \{1\} = \{1, 2, 4\}$, nên đỉnh 5 có 3 màu.

A2. Cách ngây thơ: chép tập của con lên cha

Làm đúng như công thức: tính S cho các con, rồi **tạo tập mới** cho v và chép mọi phần tử của các tập con vào. Mỗi phần tử x được chép một lần ở **mỗi tổ tiên** của nó.



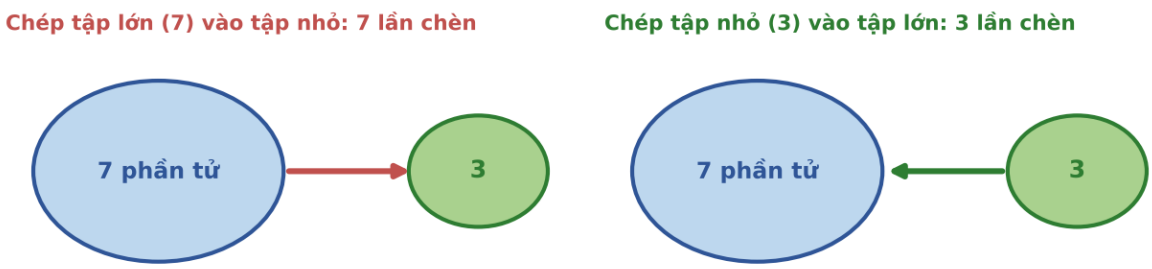
Hình 1.3. *Cây đường thẳng: chép $O(n^2)$ lần*

ĐẾM SỐ PHÉP TÍNH

- Với cây đường thẳng n đỉnh, mọi màu khác nhau: đỉnh ở độ sâu d có tập n − d phần tử; chép tập của con lên cha tốn tổng cộng $1 + 2 + \ldots + (n - 1) = n(n - 1)/2$ lần chèn.
- $n = 2 \cdot 10^5$: khoảng $2 \cdot 10^{10}$ lần chèn vào set. Quá chậm.

A3. Gộp nhỏ vào lớn

Quan sát then chốt: tập của cha **không cần là tập mới**. Ta có thể **lấy luôn một tập của con làm tập của cha** (vì tập của con không còn dùng nữa sau khi đã ghi đáp án), rồi chèn các tập còn lại vào nó. Nên giữ lại tập nào? **Tập lớn nhất**.



Hình 1.4. *Chèn tập nhỏ vào tập lớn, không làm ngược lại*

Lê Thành Nam

GỘP NHỎ VÀO LỚN

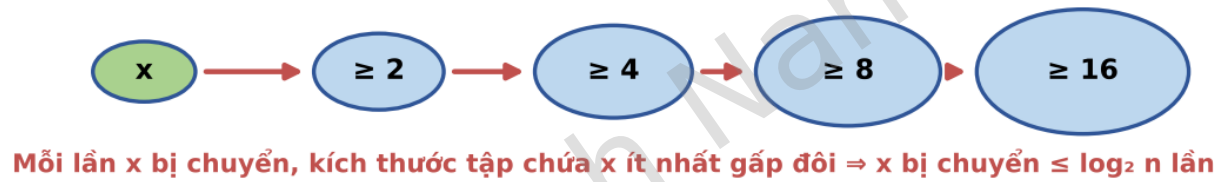
1. Mỗi đỉnh v bắt đầu với tập {c[v]}.
2. Khi gộp tập A và tập B: nếu |A| < |B| thì **đổi vai** A và B; sau đó chèn từng phần tử của B vào A; bỏ B.
3. Tập của v sau khi gộp hết các con chính là S(v).

A4. Định lí O(n log n)

SỐ LẦN CHUYỂN

Khi gộp nhỏ vào lớn, tổng số lần chèn trong mọi phép gộp **không vượt quá $n \cdot \log_2 n$**.

Ý 1 Trường hợp các màu khác nhau: kích thước tập chứa x ít nhất gấp đôi



Hình 1.5. *Trường hợp các màu khác nhau: kích thước tập chứa x ít nhất gấp đôi*

Giả sử mọi đỉnh có màu khác nhau (không có phần tử trùng). Xét một phần tử x. Mỗi khi x bị chuyển, x nằm trong tập nhỏ hơn B và được chèn vào A với $|A| \geq |B|$. Tập mới chứa x có kích thước $|A| + |B| \geq 2|B|$: **ít nhất gấp đôi** tập cũ.

Tập chứa x không bao giờ lớn hơn n. Từ kích thước 1, gấp đôi nhiều nhất $\log_2 n$ lần là vượt n. Vậy x bị chuyển không quá $\log_2 n$ lần. Có n phần tử: tổng $\leq n \cdot \log_2 n$.

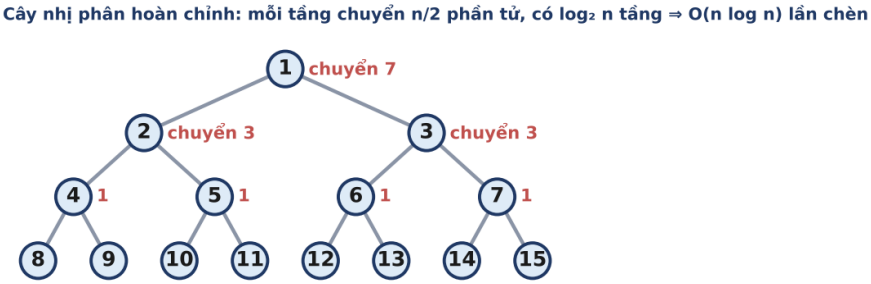
Ý 2 Trường hợp có màu trùng: đếm theo "số đỉnh đại diện"

Khi có màu trùng, $|A \cup B|$ có thể nhỏ hơn $|A| + |B|$, nên lập luận "gấp đôi" không dùng trực tiếp được. Gán cho mỗi tập một **trọng lượng** W = số đỉnh đã được gộp vào nó (W ≥ kích thước tập). Gộp hai tập thì trọng lượng cộng lại đúng: $W_{mới} = W_A + W_B$.

Chi phí một lần gộp là $\min(|A|, |B|) \leq \min(W_A, W_B)$. Trả chi phí đó bằng cách tính 1 cho **mỗi đỉnh đại diện** của phần có trọng lượng nhỏ hơn. Mỗi lần một đỉnh bị tính, trọng lượng phần chứa nó ít nhất gấp đôi, nên mỗi đỉnh bị tính $\leq \log_2 n$ lần. Tổng chi phí $\leq n \cdot \log_2 n$.

Lê Thành Nam

Ý 3 **Trường hợp xấu nhất đạt được**



Hình 1.6. *Trường hợp xấu nhất đạt được*

Với cây nhị phân hoàn chỉnh và mọi màu khác nhau, ở mỗi tầng, mọi phép gộp chuyển tổng cộng khoảng n/2 phần tử, và có $\log_2 n$ tầng. Vậy chặn n log n là chặt (sai khác hằng số).

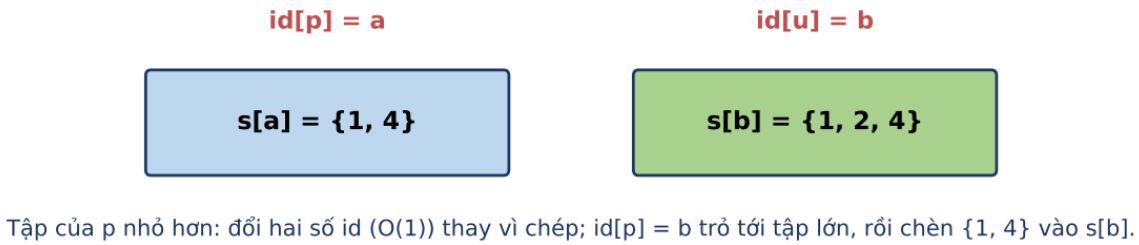
ĐẾM SỐ PHÉP TÍNH

- Mỗi lần chèn vào std::set tốn O(log n): tổng **$O(n \log^2 n)$**. Với $n = 2 \cdot 10^5$: khoảng $2 \cdot 10^5 \times 18 \times 18 \approx 6 \cdot 10^7$ phép, chạy dưới 0,5 giây.
- Dùng unordered_set: mỗi lần chèn O(1) trung bình, tổng O(n log n), nhưng hằng số lớn và dễ bị dữ liệu xấu.

A5. Cài đặt không chép, không đệ quy

Hai điều cần tránh: **chép cả tập** khi "đổi vai" (phép gán `s[a] = s[b]` tốn O(|s[b]|)), và **đệ quy** trên cây sâu. Ta dùng:

- **Mảng id[v]**: chỉ số của tập đang giữ thông tin của T(v). Đổi vai hai tập chỉ là đổi hai số id (O(1)). (std::swap của hai set cũng chỉ là O(1), nhưng id rõ ràng hơn.)
- **Duyệt ngược BFS, đẩy lên cha** như Tập 3: khi tới u, mọi con của u đã gộp xong vào tập của u: ghi đáp án |S(u)|, rồi gộp tập của u vào tập của cha.



Hình 1.7. *Đổi vai hai tập bằng hai số id*

Lê Thanh Nam

Code mẫu: Gộp nhỏ vào lớn theo ngược BFS

```
for (int k = n - 1; k >= 0; k--) {
    int u = ord[k];
    ans[u] = s[id[u]].size();
    if (u == 1) break;
    int p = par[u];
    int a = id[p], b = id[u];
    if (s[a].size() < s[b].size()) swap(a, b);
    for (int x : s[b]) s[a].insert(x);
    set<int>().swap(s[b]);
    id[p] = a;
}
```

HỘP CÔNG CỤ C++: ĐỔI VÀ GIẢI PHÓNG

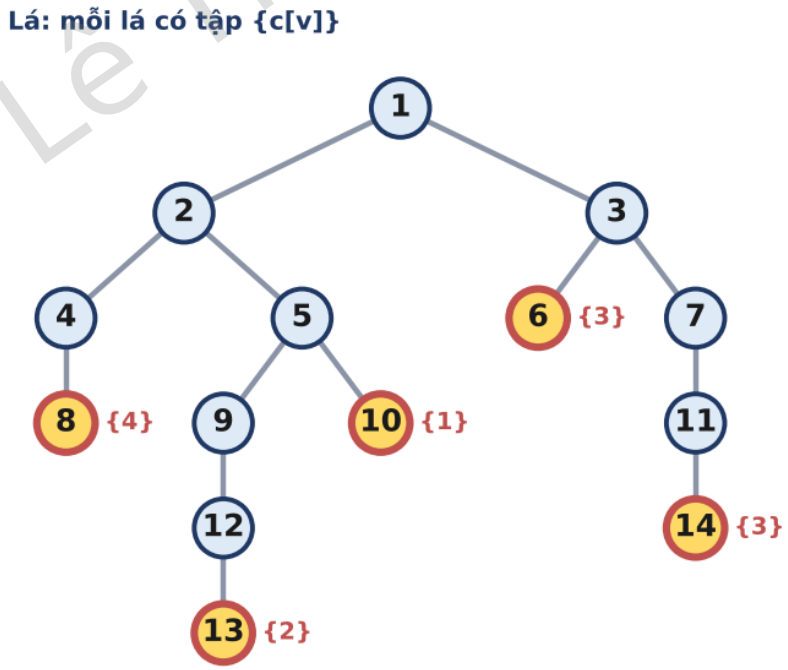
`swap(s[a], s[b])` với hai std::set (hoặc map, vector) chỉ đổi con trỏ bên trong: **O(1)**. Còn `s[a] = s[b]` là **chép**: O(kích thước).

`set<int>().swap(s[b])` đổi s[b] với một tập rỗng tạm thời; tập tạm (mang dữ liệu cũ) bị hủy ngay, nên **bộ nhớ được trả lại**. `s[b].clear()` cũng xóa phần tử, nhưng với vector thì không trả bộ nhớ.

A6. Chạy tay trên T14

Thứ tự ngược BFS: 13, 14, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2. Mỗi khung ghi tập của các đỉnh tại lúc đỉnh đó xong (đỉnh vàng: vừa xong).

Bước nhỏ 1 **Các lá**

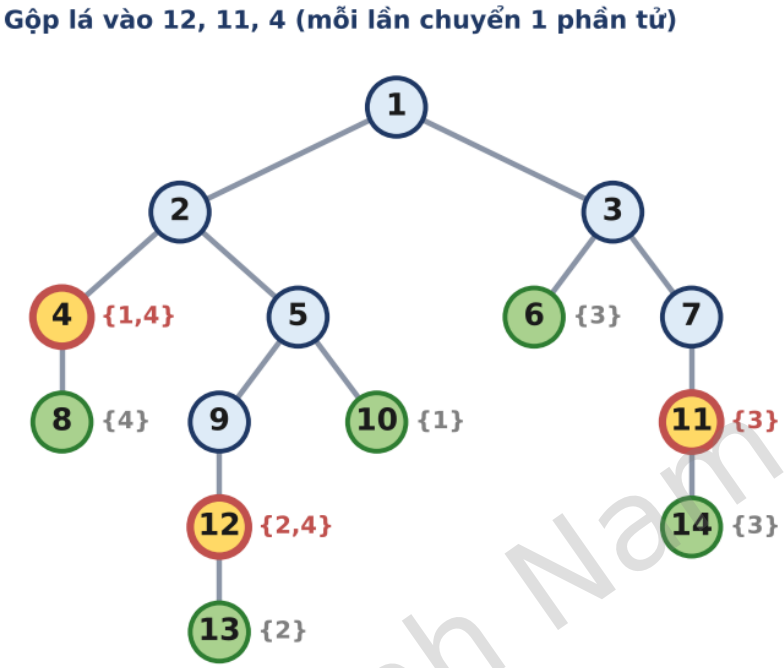


Hình 1.8. *Các lá*

Lê Thành Nam

Lá 13, 14, 8, 10, 6 có tập một phần tử: {2}, {3}, {4}, {1}, {3}.

Bước nhỏ 2 **Đỉnh 12, 11, 4**



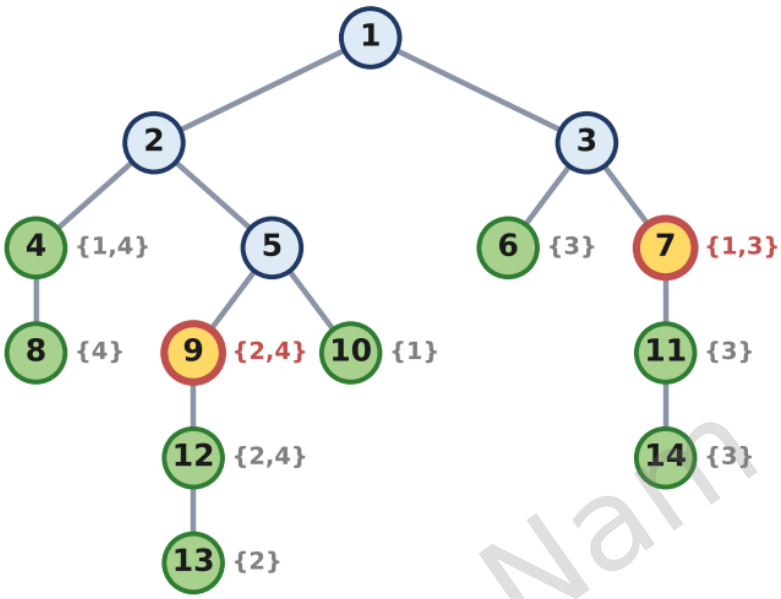
Hình 1.9. *Đỉnh 12, 11, 4*

Gộp 13 vào 12: hai tập cùng cỡ 1, chuyển 1 phần tử: S(12) = {2, 4}. Gộp 14 vào 11: {3} và {3} trùng màu: chèn 3 vào {3}, tập vẫn {3}. Gộp 8 vào 4: {1, 4}.

Lê Thành Nam

Bước nhỏ 3 **Đỉnh 9 và 7**

Gộp vào 9 và 7

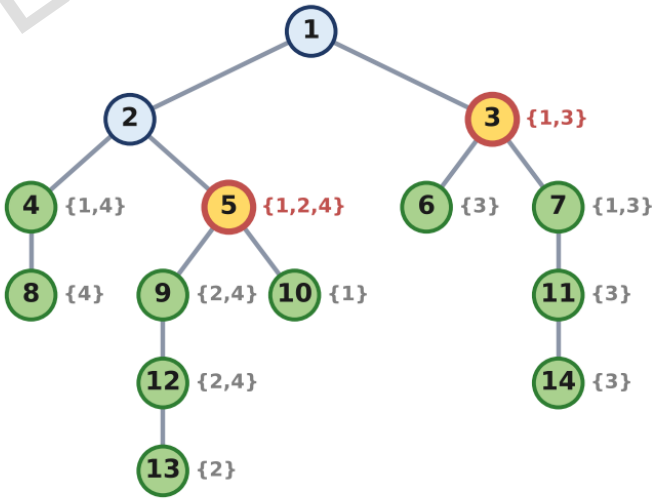


Hình 1.10. *Đỉnh 9 và 7*

Đỉnh 9: tập {2} của 9 nhỏ hơn tập {2, 4} của con 12, nên **đổi vai**: id[9] trỏ tới tập của 12, chèn 2 vào: không đổi. S(9) = {2, 4}. Đỉnh 7: tập {1} nhỏ, nhận tập {3} của 11 (cùng cỡ, không đổi vai), chèn 3: {1, 3}.

Bước nhỏ 4 **Đỉnh 5 và 3**

Đỉnh 5: gộp 10 (1 phần tử) vào tập của 9; đỉnh 3: gộp 6 vào tập của 7

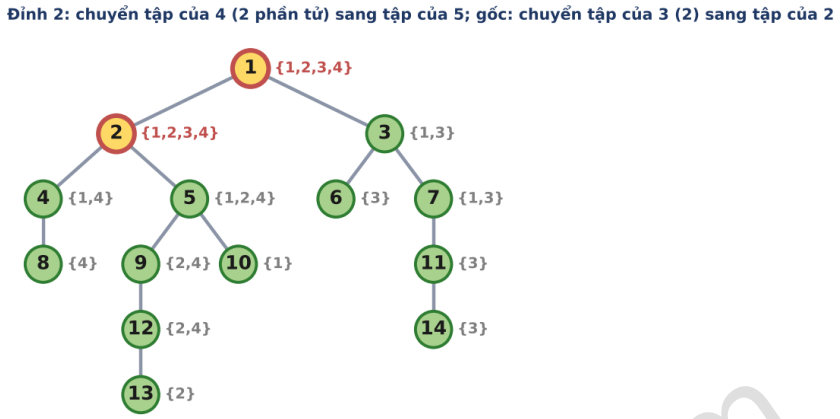


Hình 1.11. *Đỉnh 5 và 3*

Lê Thanh Nam

Đỉnh 5 có hai con 9 và 10 (thứ tự ngược BFS: 10 gộp trước). Sau khi gộp 10: {1, 2}. Gộp 9 (tập {2, 4}, cùng cỡ): chèn 2 phần tử: {1, 2, 4}, |S(5)| = 3. Đỉnh 3: gộp 7 rồi 6: {1, 3}.

Bước nhỏ 5 **Đỉnh 2 và gốc 1**



Hình 1.12. *Đỉnh 2 và gốc 1*

Đỉnh 2: nhận tập của 5 (3 phần tử) làm tập chính, chèn tập {1, 4} của 4 và màu 2: S(2) = {1, 2, 4}. Gốc: nhận tập {1, 3} của 3 trước; gặp tập {1, 2, 4} lớn hơn của 2 thì đổi vai, chèn {1, 3} vào đó: S(1) = {1, 2, 3, 4}, 4 màu.

Tổng cộng chỉ có 16 lần chèn. Nếu chép mọi tập lên cha (không gộp nhỏ vào lớn), số lần chèn bằng tổng kích thước mọi tập con, lớn hơn nhiều khi cây sâu và màu đa dạng.

A7. Liên hệ với chia nặng nhẹ

Một cách chọn khác: luôn giữ tập của **con nặng** (con có sz lớn nhất), chèn các con nhẹ vào. Mỗi phần tử chỉ bị chuyển khi đi qua một **cạnh nhẹ** trên đường về gốc, mà mỗi đường có ≤ $\log_2 n$ cạnh nhẹ (Tập 2): cũng O(n log n). Cách "so kích thước tập" ở trên linh hoạt hơn vì dùng kích thước thật của tập (có thể nhỏ hơn sz nhiều khi màu trùng). Chương 2 sẽ dùng đúng ý tưởng con nặng để bỏ hẳn các tập, chỉ dùng mảng.

BẪY THƯỜNG GẶP

- Gán `s[p] = s[u]` để "lấy tập của con": chép O(kích thước), phá hỏng chặn O(n log n). Dùng id hoặc swap.
- Quên đổi vai khi tập của cha nhỏ hơn: vẫn đúng nhưng $O(n^2)$ với cây đường thẳng có cha nhận tập lớn từ con.
- Ghi đáp án của u **trước** khi gộp đủ các con: trong cách duyệt ngược BFS, mọi con của u đứng sau u trong ord, nên đã được xử lí trước u: an toàn.
- DFS đệ quy để gộp: tràn ngăn xếp với cây đường thẳng $2 \cdot 10^5$ đỉnh.

Lê Thanh Nam

GHI NHỚ

- $S(v) = \{c[v]\} \cup$ hợp các $S(con)$. Giữ tập lớn, chèn tập nhỏ.
- Mỗi phần tử bị chuyển $\leq \log_2 n$ lần: $O(n \log n)$ lần chèn, $O(n \log^2 n)$ với set.
- id[v] + đổi chỗ O(1); duyệt ngược BFS; giải phóng tập đã dùng.

Lê Thành Nam